

Modules and Functions

Review

- ◆ **Logic constructs** = Statements can be used in a program.
 - 3 basic constructs: Sequence, selection constructs (`if`, `if...else`, `?:`), Iteration constructs (`for`/ `while`/ `do ... while`)
- ◆ **Walkthrough**
 - Code are executed by ourself
 - Tasks in a walkthrough: a record of the changes that occur in the values of program variables and listing of the output, if any, produced by the program.
 - Debug program

Objective to learn Modules and Functions

After studying this section, you should be able to:

- ◆ **Code Organization:** Programs are structured, making them easier to read and maintain.
- ◆ **Reusability:** Functions allow reuse of code, reducing duplication.
- ◆ **Debugging and Testing:** Isolated functions simplify locating and fixing bugs.
- ◆ **Maintainability:** Modular code supports easy updates and changes.
- ◆ **Abstraction:** Functions hide implementation details, focusing on functionality.
- ◆ **Scalability:** Modular design enables handling larger and more complex systems.
- ◆ **Collaboration:** Teams can work on separate modules independently.

Objective

- ◆ Define a C-module or C-function?
- ◆ Explain module's characteristics
- ◆ Implement C functions
- ◆ Use functions?
- ◆ Differentiate built-in and user-defined functions
- ◆ Explain mechanism when a function is called
- ◆ Analyze a problem into functions
- ◆ Implement a program using functions
- ◆ Understand extent and scope of a variable

Contents

- 1) What is a module?
- 2) Characteristics of modules
- 3) Hints for module identifying
- 4) C-Functions and Modules
- 5) How to implement a function?
- 6) How to use a function?
- 7) What happen when a function is called?
- 8) How to analyze a problem into functions?
- 9) Implement a program using functions
- 10) Extent and Scope of a variable
- 11) Walkthroughs with Functions

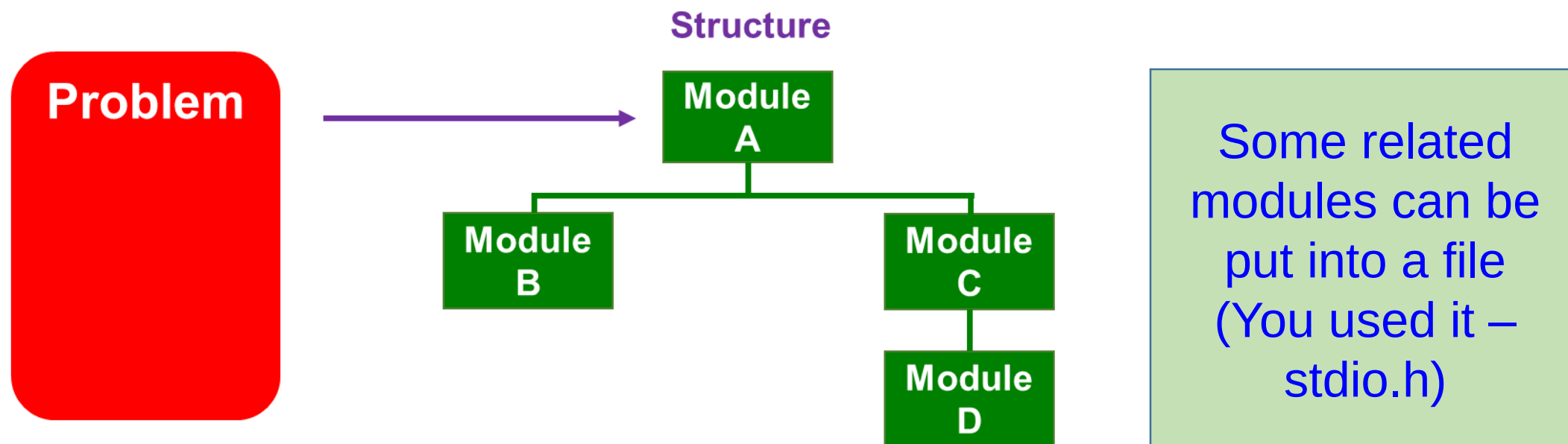
1- What is a Module?

What is a Module?

- ◆ Module is **a portion of a program** that carries out a specific small function and may be used alone or combined with other **modules** to create a program.
- ◆ Natural thinking: A large task is divided into some smaller tasks.
- ◆ Example: To cook rice we divide it into small tasks
(1) Clean the pot; (2) Measure rice; (3) Washing rice; (4) add water;
(5) Boil; (6) Keep hot 10 minutes.

Modules: Structure Design

- ◆ In designing a program, we subdivide the problem conceptually into a set of design units. We call these design units as **modules**. In subdividing the problem, we reduce the number of factors with which to deal simultaneously.



Structure design - Example

- ◆ Problem: Develop a program that will accept a positive integer then sum of it's divisors is printed out.
- ◆ Solution:

Analyze	Code	Description
	<code>#include <stdio.h></code> <code>#include <stdlib.h></code>	Use modules in this file
Divide the program into small tasks:	<code>int main</code> <code>{</code> <code> int n; int s;</code>	Declare the main module and it's data
Task 1 - Accept n	<code>scanf("%d", &n);</code>	Use a module scanf in the stdio.h
Task 2 - s = sum of it's divisors	<code>s = sumDivisors (n);</code>	Module will be implemented
Task 3 - Print out s	<code>printf("%d", s);</code>	Use a module printf in the stdio.h
Task 4 - Pause the program	<code>system("pause");</code>	Use a module system in the stdlib.h
	<code>}</code>	

2 - Characteristics of Modules

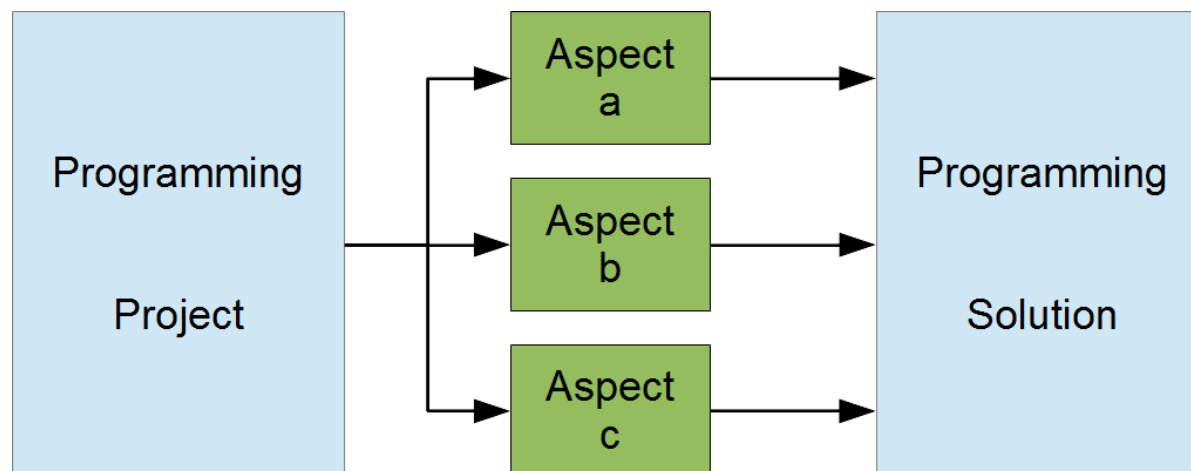
Characteristics of Modules

Characteristics	Reason
It is easy to upgrade and maintain	It contains a small group of code lines for a SPECIFIC task.
It can be re-used in the same program	It has a identified name (a descriptive identifier) and can be used more than one time in a program.
It can be re-used in some programs	if it is stored in an outside file (library file), it can be used in some programs.

3 - Module identifying: Hints

Modules Design Principles

- ◆ We can sub-divide a programming project in different ways.
- ◆ Select our modules so that each one focuses on a narrower aspect of the project.
- ◆ Our objective is to define a set of modules that simplifies the complexity of the original problem.



Modules Design Principles (cont.)

- ◆ **Some general guidelines for defining a module include:**
 - The module is easy to upgrade
 - The module contains a readable amount of code
 - The module may be used as part of the solution to some other problem
- ◆ **For a structured design, we stipulate that:**
 - Each module has one entry point and one exit point
 - Each module is **highly cohesive**
 - Each module exhibits **low coupling**

Module Identifying

Lowly cohesive

An input operation
in a processing
module is not
encouraged.

→ All the code in a
module focus to the
purpose of the
module

```
#include <stdio.h>
```

```
int n ;
```

Module for summing divisors of n

```
{ accept n  
  sum of it's divisors  
}
```

Module for printing out divisors of n

```
{ accept n  
  print out it's divisors  
}
```

```
int main ()
```

```
{  
  access n  
}
```

High coupling

Some modules
access a common
data is not
encouraged.

→ All modules
should be self-
contained
(independent)

Module identifying - Cohesion

- ◆ Cohesion is a measure of the focus within a module.
- ◆ A module performs a single task → **highly cohesive**.
- ◆ A module performs a collection of unrelated tasks → **low cohesion**.
- ◆ In designing a cohesive module, we ask whether a certain task belongs:
 - The reason to include it is that it is related to the other tasks in some particular manner.
 - A reason to exclude it is that it is unrelated to the other tasks.
- ◆ How to identify modules? → **If you still use a verb to describe a task then a module is identified.**

Module identifying - Degrees of cohesion

Low cohesion → generally unacceptable

- ◆ **Coincidental** - unrelated tasks
→ This module is not enough small → Separate smaller tasks in this task.
- ◆ **Logical** - This module contains some related tasks of which only one is performed
→ Separate them into separate smaller module, each smaller module for a choice.
- ◆ **Temporal** - multiple logically unrelated tasks that are only temporally related
→ Separate them into separate smaller module although they are temporal

One module for two tasks:

- Sum divisors of the integer n
- Print out divisors of the integer n

In the case of the operation for summing of n is not used, this module can not be applied.

Module identifying - Degrees of cohesion

High cohesion - generally acceptable

- ◆ **Communicational** - the tasks share the same data
 - All tasks are carried out each time.
- ◆ **Sequential** - multiple tasks in a sequentially dependent relationship.
 - Output of one task serves as input to another task - the module identifier suggests an assembly line.
- ◆ **Functional** - performs a single specific task
 - The module identifier suggests a precise verb phrase

Some modules share the common data can be accepted if they perform their tasks sequentially.

Module identifying - High Coupling

High Coupling - It's not advisable

- ◆ **Coupling** is a measure of the **degree of interrelatedness** of a module to its referring module(s).
- ◆ A module is low in coupling if it performs its tasks on its own.
- ◆ A module is **highly coupled** if it shares that performance with some other module including the referring module → **It should not be used**.
- ◆ In designing for low coupling, we ask what kind of data to avoid passing to the module.

Module identifying - Coupling classification

- ◆ The data classifications include:

high

- **Data** - used by the module but not to control its execution → **Data in/dependant**

- **Control** - controls the execution of the module → **Control in/dependent**

- **External** - part of an environment external to the module that controls its execution

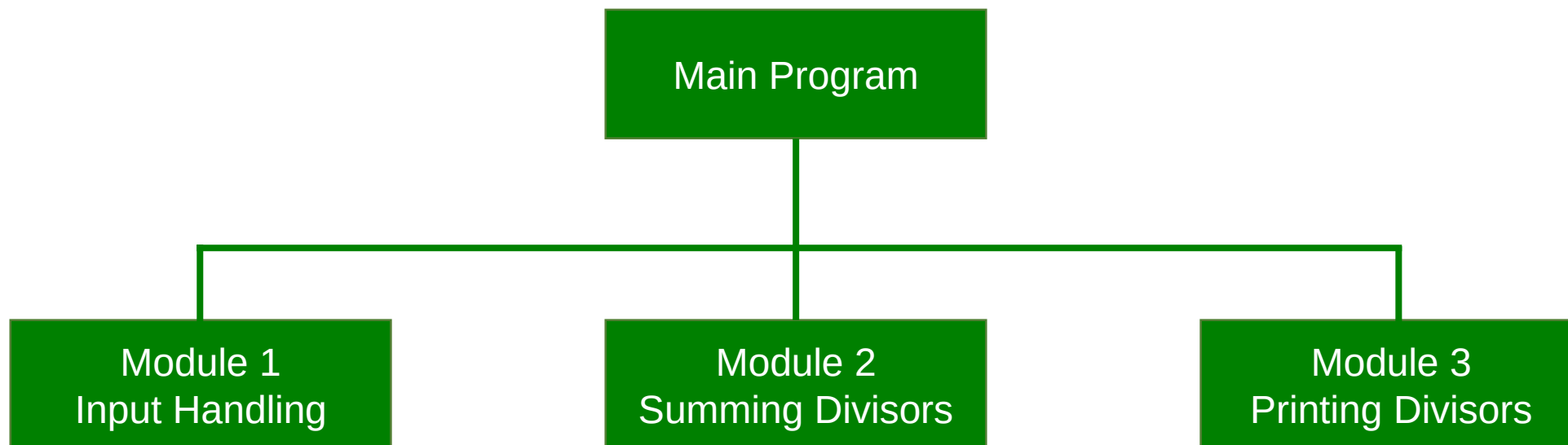
- **Common** - part of a global set of data

low

- **Content** - accesses the internals of another module

Example - High Cohesion & Low Coupling

- ◆ Problem: Develop a program that will accept a positive integer then sum of it's divisors is printed out.
- ◆ Modules design:



Example - High Cohesion & Low Coupling (cont.)

- ◆ **Module 1: Input Handling**
 - This module is responsible for reading the value of **n** from the user.
- ◆ **Module 2: Summing Divisors**
 - This module calculates the sum of all divisors of a given integer.
- ◆ **Module 3: Printing Divisors**
 - This module prints all divisors of a given integer.
- ◆ **Main Program**
 - This module coordinates the other modules. It uses input, calculates the sum of divisors, and prints the divisors.

Example - High Cohesion & Low Coupling (cont.)

```
module_demo.c
1  #include <stdio.h>
2
3  // Module 1: Input Handling
4  int getInput() {
5      int n;
6      printf("Enter a number: ");
7      scanf("%d", &n);
8      return n;
9  }
10
11 // Module 2: Summing Divisors
12 int sumOfDivisors(int n) {
13     int sum = 0;
14     for (int i = 1; i <= n; i++) {
15         if (n % i == 0) {
16             sum += i;
17         }
18     }
19     return sum;
20 }
```



```
22 // Module 3: Printing Divisors
23 void printDivisors(int n) {
24     printf("Divisors of %d are: ", n);
25     for (int i = 1; i <= n; i++) {
26         if (n % i == 0) {
27             printf("%d ", i);
28         }
29     }
30     printf("\n");
31 }
32
33 // Main Program
34 int main() {
35     // Input
36     int n = getInput();
37
38     // Processing
39     int sum = sumOfDivisors(n);
40
41     // Output
42     printf("The sum of divisors of %d is: %d\n", n, sum);
43     printDivisors(n);
44
45     return 0;
46 }
```

Output

```
D:\MonHoc\PRF192\ThucHanh\module_demo.exe
Enter a number: 12
The sum of divisors of 12 is: 28
Divisors of 12 are: 1 2 3 4 6 12

-----
Process exited after 19.82 seconds with return value 0
Press any key to continue . . .
```

Example Explain - High Cohesion & Low Coupling

- ◆ **High Cohesion:** Each module has a single, well-defined purpose
 - **getInput:** Handles user input.
 - **sumOfDivisors:** Calculates the sum of divisors.
 - **printDivisors:** Displays all divisors.
- ◆ **Low Coupling:**
 - Each module operates independently and communicates only via function calls and return values.
 - No global variables are shared between modules.
- ◆ **Reusability:** Each module can be reused in other programs without modification
- ◆ **Ease of Maintenance:** Modifications to one module (e.g., changing the input method) won't affect others.

Module identifying : How to create them?

- ◆ If you still use a verb to describe a task then a module is identified.
- ◆ In practice:
 - List all of the tasks (verbs) that the program should perform to solve this problem.
 - Identify the modules (verbs) for the problem structure
 - Check that each module is **high in cohesion** (each basic task is a module)
 - Check that each module is **low in coupling** (modules are independent)

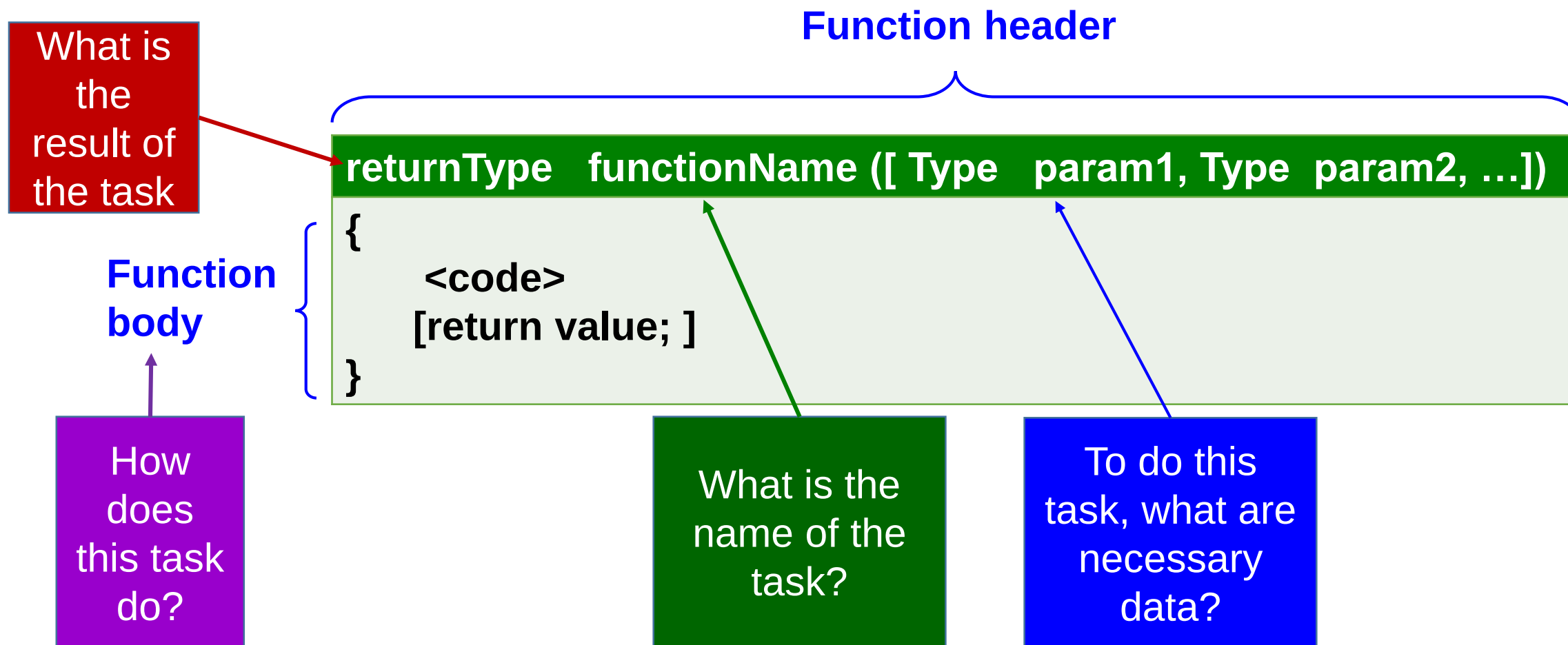
4 - C-Functions and Modules

C-Functions and Modules

- ◆ In C, we represent a module by a function.
- ◆ A function is a block of reusable code designed to perform a specific task. A function may receive data and may return a value.
- ◆ Example:
 - Print out divisors of the integer $n \rightarrow n$ is data is accepted by the function and no value is returned.
 $n = 12 \rightarrow$ Print out values: 1, 2, 3, 4, 6, 12
 - Sum of divisors of the integer $n \rightarrow n$ is data is accepted by the function and a value is returned.
 $n = 12 \rightarrow 28$ is the return value
- ◆ The description of the internal logic of a function as the function's definition.

Function Definitions

- ◆ Function Syntax:



Function Definitions - Example

return DataType

Function Identifier

Parameters

double **average** (int a, int b, int c)

{

double result;
result = (a+b+c)/3. ;
return result;

}

Body:
Logical construct

Review:

(a+b+c)/3 → integer

Review:

(a+b+c)/3.0 → double

Review

3.0 and 3. are the same
3.3500 = 3.35
3.30 = 3.3
3.0 = 3.

Function Definitions - Example (cont.)

```
function_syntax.c
1  #include <stdio.h>
2
3  /*
4   * Function: average
5   * -----
6   * Computes the average of 3 integers
7   * parameters: a, b, c are integer numbers
8   * returns: the average of 3 integers
9   */
10 double average(int a, int b, int c)
11 {
12     double result;
13     result = (a+b+c)/3.;
14     return result;
15 }
16
17 int main(){
18     // Implementing function
19     double avgNumbers = average(5, 8, 7);
20     printf("The average of the numbers: %lf", avgNumbers);
21
22     return 0;
23 }
```



```
D:\MonHoc\PRF192\ThucHanh\function_syntax.exe
The average of the numbers: 6.666667
-----
Process exited after 0.0824 seconds with
return value 0
Press any key to continue . . .
```

Function syntax: void function

- ◆ To identify a function that does not return any value, we specify **void** keyword for the return data type and exclude any expression from the return statement:
 - Alternatively, we can omit the **return** statement altogether.
- ◆ A function that does not return a value is called a subroutine or procedure in other languages.
- ◆ Syntax:

```
void functionName ([ Type param1, Type param2, .... ] )  
{  
    // Statements  
}
```

void function - Example

```

1  /* Example 1: Count down an integer n to 1
2  *  countDown.c
3  */
4
5  #include <stdio.h>
6
7  // void function with parameter
8  void countDown(int n){
9      while (n > 0)
10     {
11         printf("%d ", n);
12         n--;
13     }
14 }
15
16 int main(){
17     // Inplemeting function
18     countDown(10);
19
20     return 0;
21 }

```

10 9 8 7 6 5 4 3 2 1

Process exited after 0.09705 seconds with return value 0
Press any key to continue . . .

```

1  /* Example 2: Alphabet A - Z
2  *  alphabet.c
3  */
4
5  #include <stdio.h>
6
7  // void function without parameter
8  void alphabet()
9  {
10     char letter = 'A';
11
12     do {
13         printf("%d ", letter);
14         letter++;
15     } while (letter != 'Z');
16 }
17
18 int main(){
19     // Inplemeting function
20     alphabet();
21
22     return 0;
23 }

```

65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89

Process exited after 0.08573 seconds with return value 0
Press any key to continue . . .

main function

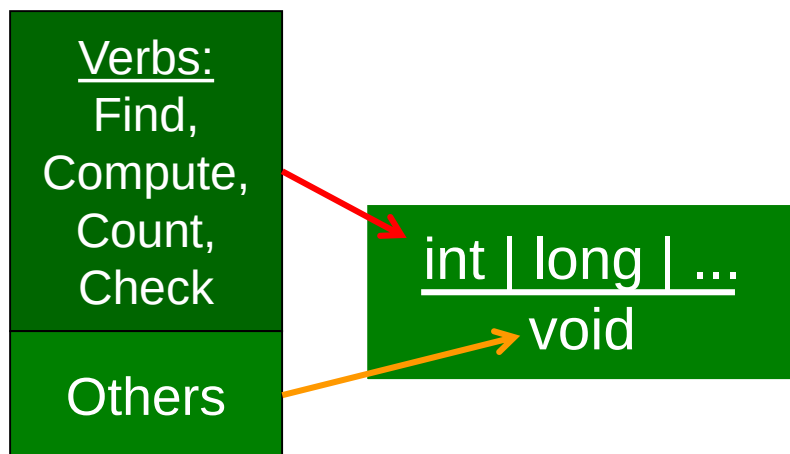
- ◆ The **main()** function is the function to which the operating system transfers control at the start of execution.
- ◆ **main()** returns a value to the operating system upon completing execution. C compilers assume an **int** where we don't provide a return data type.
- ◆ The operating system typically accepts a value of **0** as an indicator of success and may use this value to control subsequent execution of other programs.
- ◆ **main() is the entry point of a C- program**

```
1  #include <stdio.h>
2
3  // definition other functions
4
5  int main(){
6      // statements
7
8      return 0;
9  }
```

5 - How to implement a function?

How to implement a function?

State the task clearly: Verb + nouns (Objects)



```
functionName(Type param1, Type param2)
{
    <steps of processing>
    return [ Expression];
}
```

Give values to the parameters;
Carry out the work with yourself;
Write down steps;
Translate steps to C;

A task is described clearly if the receiver does not need to ask any thing

Evaluate the functions

This function contains a sub-task
→ low cohesive.

```
#include <stdio.h>
// Test whether an integer is a prime or not
int isPrime(int n){
    printf("Input n=");
    scanf("%d", &n);
    int i;
    for(i=2; i*i<=n; i++){
        if(n%i==0){
            return 0;
        }
    }
    return 1;
}

int main(){
    int n;
    if(isPrime(n)==1){
        printf("n is prime.\n");
    }else{
        printf("n is not prime.");
    }
    return 0;
}
```

fix
→

Better

```
#include <stdio.h>
// Test whether an integer is a prime or not
int isPrime(int n){
    int i;
    for(i=2; i*i<=n; i++){
        if(n%i==0){
            return 0;
        }
    }
    return 1;
}

int main(){
    int n;
    printf("Input n=");
    scanf("%d", &n);
    if(isPrime(n)==1){
        printf("n is prime.\n");
    }else{
        printf("n is not prime.\n");
    }
    return 0;
}
```

Functions for testing will return 1 for true and 0 for false.

Common algorithm in testing is checking all cases which cause FALSE. TRUE is accept when no case cause FALSE

Evaluate the functions

This function accesses outside data
→ rather coupling

```
#include <stdio.h>

int a=5, b=10, c=2;
// Computes the average of 3 integer numbers
double average(){
    return (a+b+c)/3.0;
}

int main(){
    printf("Average of 3 integer numbers: %.2lf", average());
    return 0;
}
```

fix

```
#include <stdio.h>
// Computes the average of 3 integer numbers
double average(int a, int b, int c){
    return (a+b+c)/3.0;
}
int main(){
    int x = 5, y = 10, z = 2;
    printf("Average of 3 integer numbers: %.2lf", average(x,y,z));
    return 0;
}
```

Better

6 - How to use a function?

How to use a function?

- ◆ In C, you can use either the built-in library functions or your own functions.
- ◆ If you use the built-in library functions, your program needs to begin with the necessary include file.
- ◆ Syntax for using a function:

```
functionIdentifier (argument1, argument2, ...);
```

- ◆ Distinguish parameters and arguments
 - **Parameters:** names of data in function implementation
 - **Arguments:** data used when a function is called

Practice 1

- ◆ Develop a program that will perform the following task in three times:
 - Accept a positive integer.
 - **Print out it's divisors**
- ◆ Analyze: Print out divisors of the positive integer n

```
n=6
i=1 → n%i → 0 → Print out i
i=2 → n%i → 0 → Print out i
i=3 → n%i → 0 → Print out i
i=4 → n%i → 1
i=5 → n%i → 1
i=6 → n%i → 0 → Print out i
```

solution

```
for i=1 ... n
    if (n%i ==0) print out i;
```

User-defined function

```
void printDivisors(int n)
{
    int i;
    for ( i=1; i<=n; i++)
        if (n%i==0) printf("%d, ", i );
}
```


Practice 1 - Solution

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(){
5      int n, i;
6
7      printf("\nInput n=");
8      scanf("%d", &n);
9      for(i=1; i<=n; i++){
10         if(n%i==0){
11             printf("%d, ", i);
12         }
13     }
14
15     printf("\nInput n=");
16     scanf("%d", &n);
17     for(i=1; i<=n; i++){
18         if(n%i==0){
19             printf("%d, ", i);
20         }
21     }
22
23     printf("\nInput n=");
24     scanf("%d", &n);
25     for(i=1; i<=n; i++){
26         if(n%i==0){
27             printf("%d, ", i);
28         }
29     }
30
31     printf("\n");
32     system("pause");
33     return 0;
34 }
```

Same output

```

D:\MonHoc\PRF192\ThucHanh\practi...
Input n=6
1, 2, 3, 6,
Input n=5
1, 5,
Input n=10
1, 2, 5, 10,
Press any key to continue . . .
```

A function can be re-used.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void printDivisors(int n){
5      int i;
6      for(i=1; i<=n; i++){
7          if(n%i==0){
8              printf("%d, ", i);
9          }
10     }
11 }
12
13 int inputN(){
14     int n;
15     printf("\nInput n=");
16     scanf("%d", &n);
17     return n;
18 }
19
20 int main(){
21     int i;
22
23     for(i=1; i<=3; i++){
24         int n;
25         n = inputN();
26         printDivisors(n);
27     }
28
29     printf("\n");
30     system("pause");
31     return 0;
32 }
```

parameter

User-defined
function

Call
function

argument

repeat

What do you think if the program will perform this task 20 times?

Exercise 1

- ◆ Develop a program that will accept a positive integer then sum of it's divisors is printed out.
- ◆ Requirement: Implementation of User-defined function
- ◆ *Hint:* **Sum of divisors of the positive integer n**

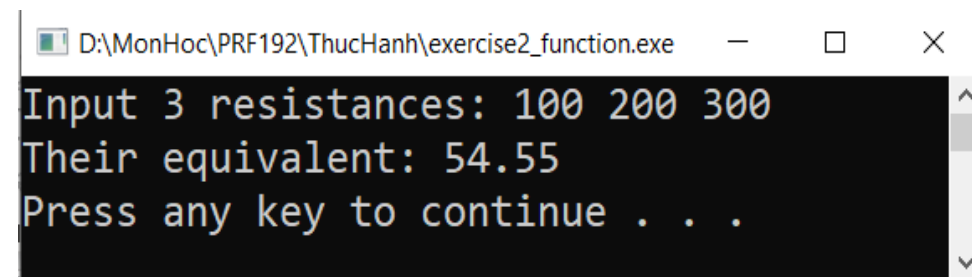
```
n=6, S=0
i=1 → n%i → 0 → S= 0+1=1
i=2 → n%i → 0 → S= 1+2=3
i=3 → n%i → 0 → S= 3+3=6
i=4 → n%i → 1
i=5 → n%i → 1
i=6 → n%i → 0 → S=6+6=12
```

Exercise 2

- ◆ Develop a program that will accept 3 parallel circuit resistances and their equivalent is printed out.
- ◆ Complete the program below:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  double equivalent(double r1, double r2, double r3){
5      // Your code
6  }
7
8  int main(){
9      double R1, R2, R3, Z;
10
11     printf("Input 3 resistances: ");
12     scanf("%lf%lf%lf", &R1, &R2, &R3);
13
14     printf("Their equivalent: %.2lf\n", equivalent(R1, R2, R3));
15
16     system("pause");
17     return 0;
18 }
```

$$1/Z = 1/r1 + 1/r2 + 1/r3 \rightarrow Z = ?$$



```
D:\MonHoc\PRF192\ThucHanh\exercise2_function.exe
Input 3 resistances: 100 200 300
Their equivalent: 54.55
Press any key to continue . . .
```

Function Prototypes

- ◆ Function prototypes describe the form of a function without specifying the implementation details
- ◆ Function prototypes declaration is put at a place and it's implementation is put at other.
- ◆ **Syntax:** `returnType functionName ([Type1 param1, Type2 param2, ... ]);`
- ◆ When the program is compiled:
 - Step 1: The compiler acknowledges this prototype (return type, name, order of data types in parameters) and marks places where this function is used and continues the compile process.
 - Step 2: If the function is detected, the compiler will update the marks in the previous step to create the program. Else, an error is thrown.

Function Prototypes - Example

```

1  /* prototypes.c
2  *   Computes the odd number: 1 -> n
3  *   Author: ThoPN3
4  *   Date: yyyy/MM/dd
5  */
6  #include <stdio.h>
7
8  int main(){
9      int n, result;
10
11      printf("Input n=");
12      scanf("%d", &n);
13
14      result = sumOddNumbers(n);
15      printf("Sum the odd numbers is: %d", result);
16
17      return 0;
18 }
19
20 int sumOddNumbers(int n){
21     int i, sum = 0;
22     for(i=1; i<=n; i++){
23         if(i%2!=0){
24             sum += i;
25         }
26     }
27     return sum;
28 }

```

```

1  /* prototypes.c
2  *   Computes the odd number: 1 -> n
3  *   Author: ThoPN3
4  *   Date: yyyy/MM/dd
5  */
6  #include <stdio.h>
7
8  // Function Prototype definition
9  int sumOddNumbers(int n);
10
11 int main(){
12     int n, result;
13
14     printf("Input n=");
15     scanf("%d", &n);
16
17     result = sumOddNumbers(n);
18     printf("Sum the odd numbers is: %d", result);
19
20     return 0;
21 }
22
23 int sumOddNumbers(int n){
24     int i, sum = 0;
25     for(i=1; i<=n; i++){
26         if(i%2!=0){
27             sum += i;
28         }
29     }
30     return sum;
31 }

```

Compiler (2) Resources Compile Log Debug Find Results Console Close			
Line	Col	File	Message
14	11	D:\MonHoc\PRF192\ThucHanh\prototype.c	[Warning] implicit declaration of function 'sumOddNumbers' [-Wimplicit-function-declaration]

Input n=10
Sum the odd numbers is: 25

The `#include` directive

- ◆ We use the `#include` directive to instruct the compiler to insert a copy of the header file into our source code.

- ◆ Syntax: `#include "filename" // in user directory`
`#include <filename> // in system directory`

- ◆ Example:

myFuntions.c

```
1 int sumOddNumbers(int n){
2     int i, sum = 0;
3     for(i=1; i<=n; i++){
4         if(i%2!=0){
5             sum += i;
6         }
7     }
8     return sum;
9 }
```

```
#include <stdio.h>
#include "myFuntions.c"

int main(){
    int n, result;

    printf("Input n=");
    scanf("%d", &n);

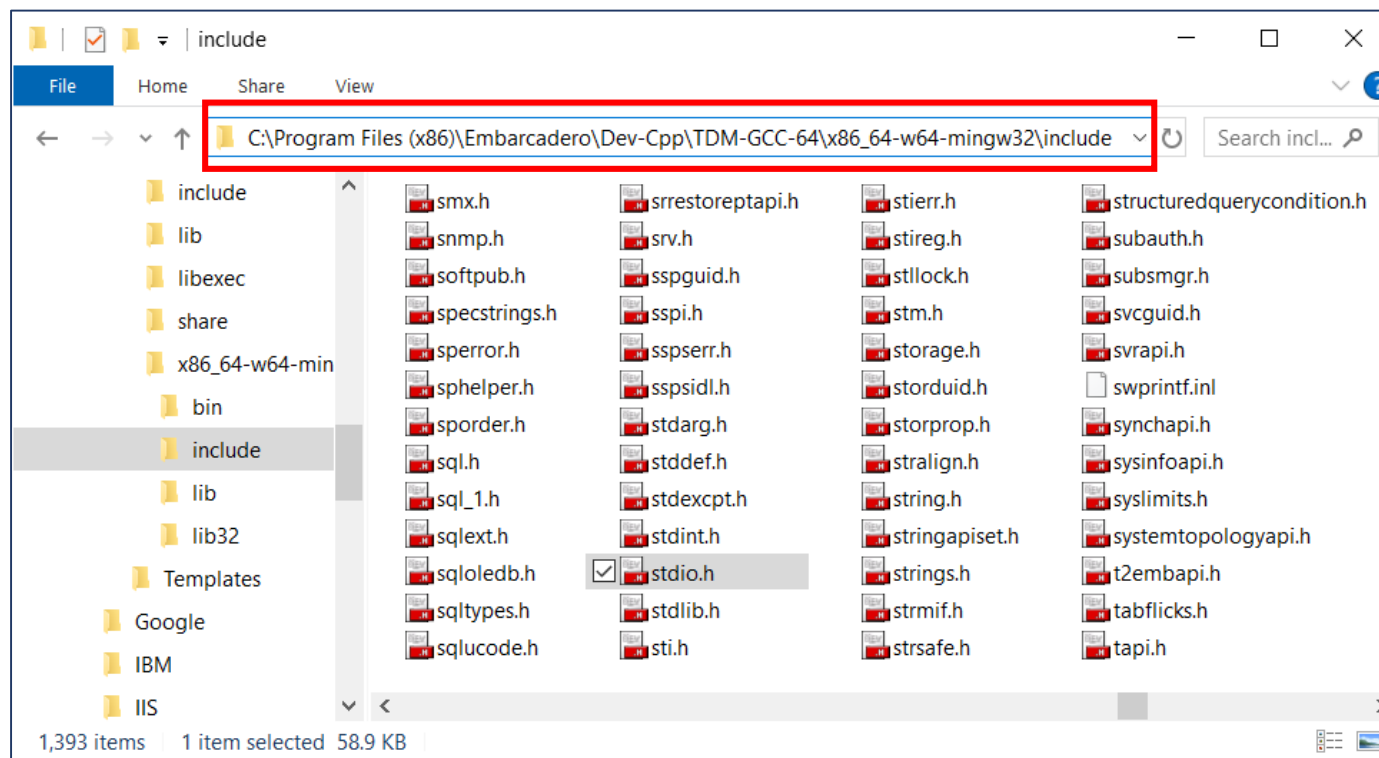
    result = sumOddNumbers(n);
    printf("Sum the odd numbers is: %d", result);

    return 0;
}
```

Input n=10
Sum the odd numbers is: 25

The **#include** directive (cont.)

- ◆ System directory: The **include** directory of the select programming environment (such as Dev-Cpp)



Function Programming Style

For style, we

- ◆ Declare a prototype for each function definition
- ◆ Specify the return data type in each function definition
- ◆ Specify void for a function with no parameters
- ◆ Avoid calling the main function recursively
- ◆ Include parameter identifiers in the prototype declarations
- ◆ Use generic comments and variables names so that we can use the function in a variety of applications without modifying its code

7 - What happen when a function is called?

Memory map when a function is called

```

1  #include <stdio.h>
2
3  int myVar = 10;
4
5  double average(int a, int b){
6      double result;
7      result = (a+b)/2.0;
8
9      printf("\nIn average function\n");
10     printf("%-15s %-15s %-15s\n", "Name", "Address", "Value");
11     printf("-----\n");
12     printf("%-15s %-15u %-15d\n", "a", &a, a);
13     printf("%-15s %-15u %-15d\n", "b", &b, b);
14     printf("%-15s %-15u %-15lf\n", "result", &result, result);
15
16     return result;
17 }
18
19 int main(){
20     int a=5, b=8;
21
22     printf("In main function\n");
23     printf("%-15s %-15s %-15s\n", "Name", "Address", "Value");
24     printf("-----\n");
25     printf("%-15s %-15u %-15d\n", "myVar", &myVar, myVar);
26     printf("%-15s %-15u %-15d\n", "a", &a, a);
27     printf("%-15s %-15u %-15d\n", "b", &b, b);
28
29     printf("Address of main(): %u\n", &main);
30     printf("Address of average(...): %u\n", &average);
31     printf("Result returned to main: %lf", average(a, b));
32
33     return 0;
34 }
    
```

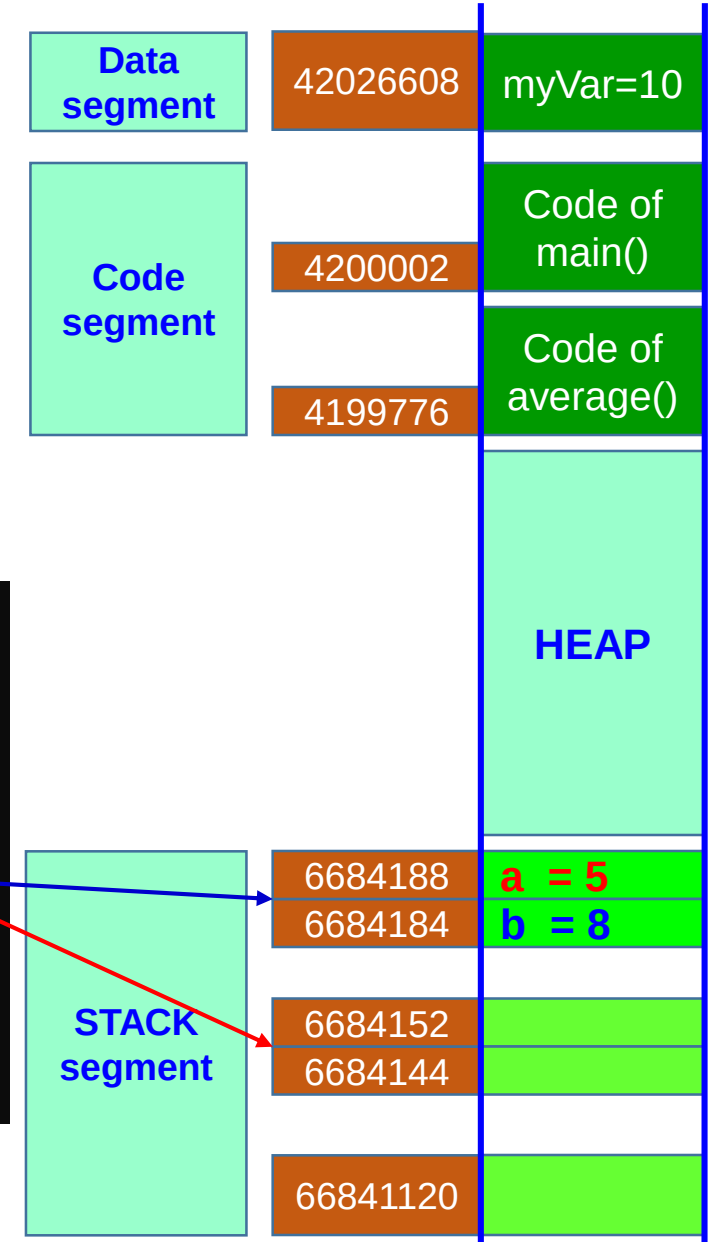
Memory mapping

In main function

Name	Address	Value
myVar	4206608	10
a	6684188	5
b	6684184	8
Address of main(): 4200002		
Address of average(...): 4199776		

In average function

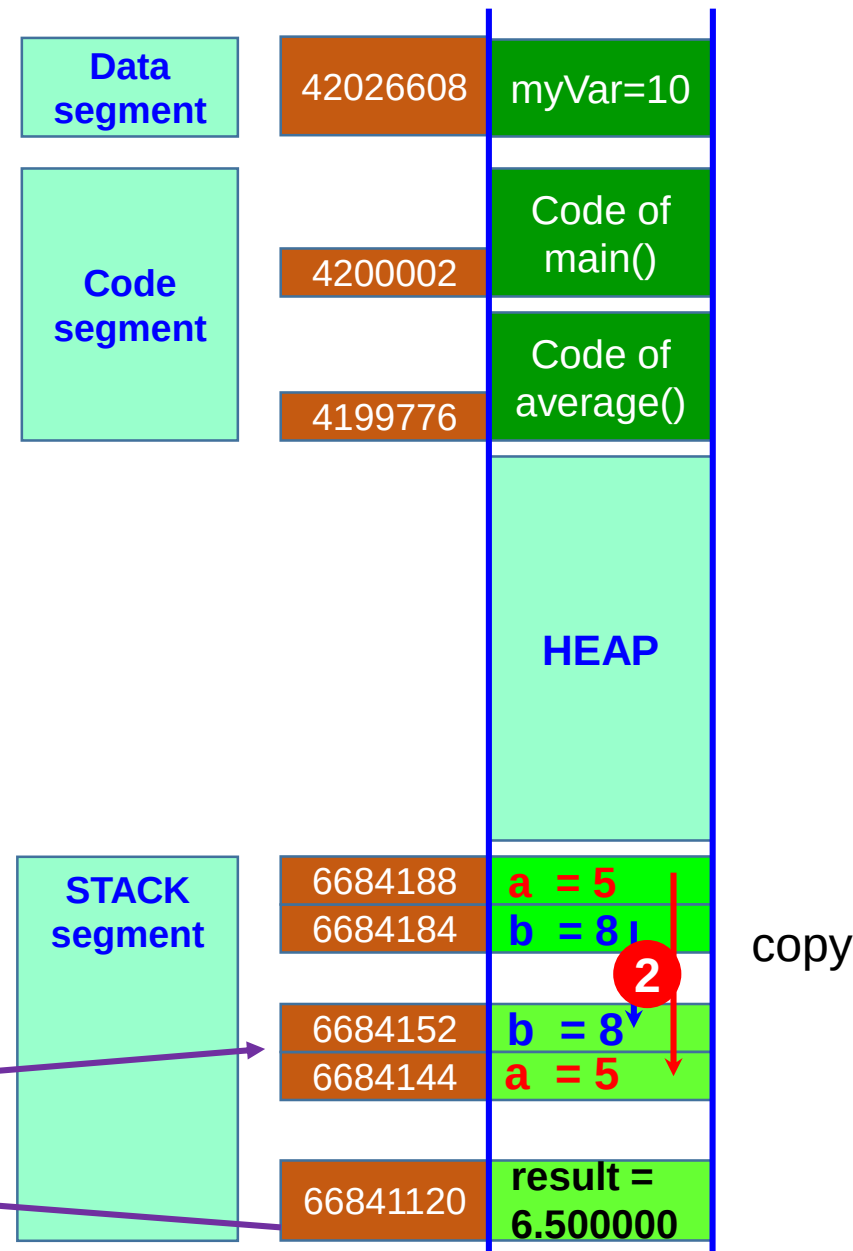
Name	Address	Value
a	6684144	5
b	6684152	8
result	6684120	6.500000
Result returned to main: 6.500000		



Memory map when a function is called

```

1  #include <stdio.h>
2
3  int myVar = 10;
4
5  double average(int a, int b){
6      double result;
7      result = (a+b)/2.0;
8
9      printf("\nIn average function\n");
10     printf("%-15s %-15s %-15s\n", "Name", "Address", "Value");
11     printf("-----\n");
12     printf("%-15s %-15u %-15d\n", "a", &a, a);
13     printf("%-15s %-15u %-15d\n", "b", &b, b);
14     printf("%-15s %-15u %-15lf\n", "result", &result, result);
15
16     return result;
17 }
18
19 int main(){
20     int a=5, b=8;
21
22     printf("In main function\n");
23     printf("%-15s %-15s %-15s\n", "Name", "Address", "Value");
24     printf("-----\n");
25     printf("%-15s %-15u %-15d\n", "myVar", &myVar, myVar);
26     printf("%-15s %-15u %-15d\n", "a", &a, a);
27     printf("%-15s %-15u %-15d\n", "b", &b, b);
28
29     printf("Address of main(): %u\n", &main);
30     printf("Address of average(...): %u\n", &average);
31     printf("Result returned to main: %lf", average(a, b));
32
33     return 0;
34 }
    
```



Pass by value

- ◆ C-language uses the “**pass by value**” only when passing arguments to called functions.
- ◆ The function receives copies of the data supplied by the arguments in the function call (*The compiler allocates space for each parameter and initializes each parameter to the value of the corresponding argument in the function call*).
- ◆ So, anything passed into a function call is unchanged in the caller's scope when the function returns
 - Parameters and arguments stored in different addresses
 - Although they have the same names, they are still different
- ◆ When a called function completes its task, its memory block, allocated, is de-allocated.

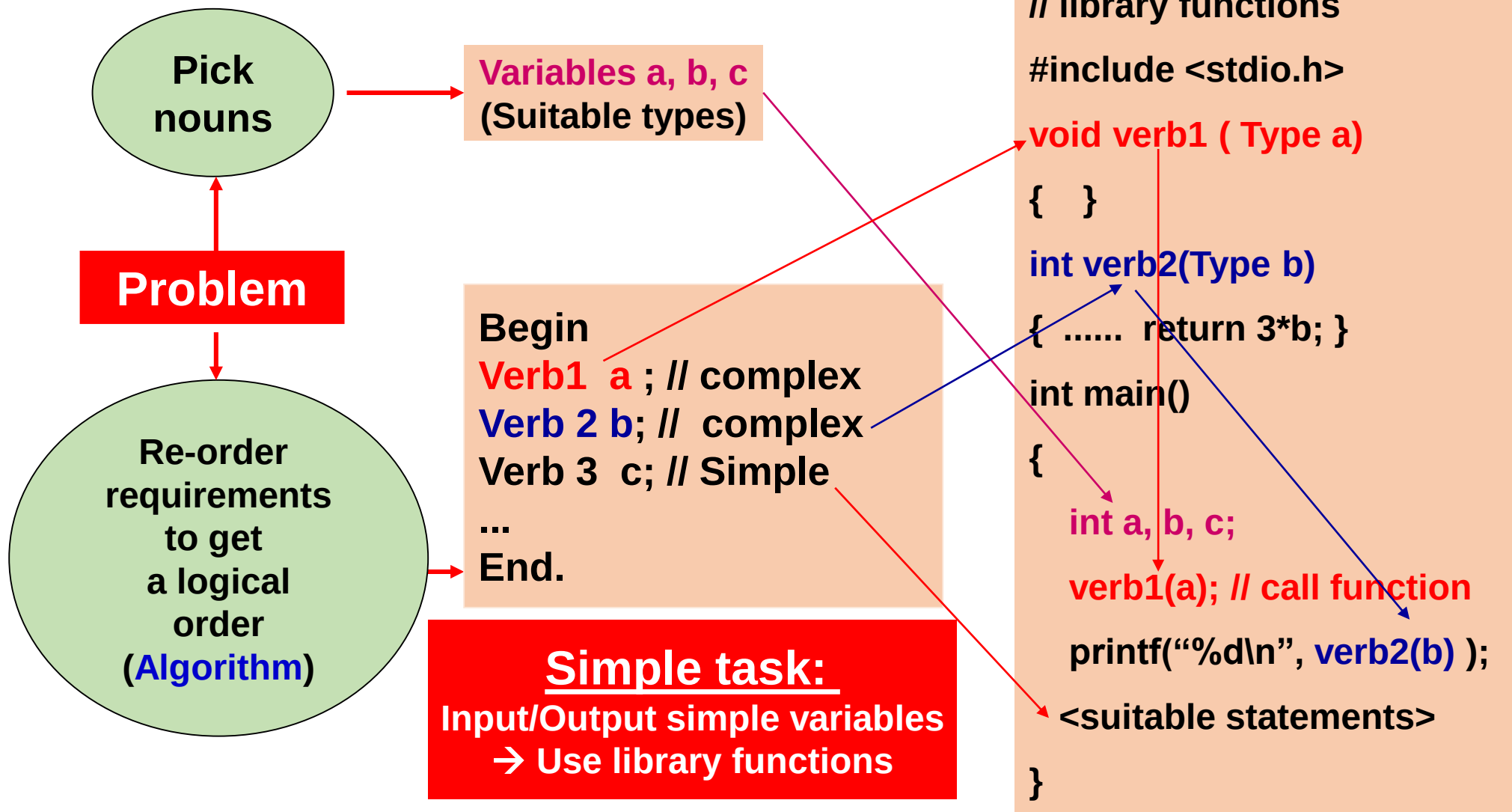
Exercise 3

- ◆ A program for swapping two integers is implemented as this code.
- ◆ Rewrite, compile and run this program.
 - Draw memory map
 - Explain the result

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void swap( int a, int b)
5  {   int t;
6      printf("In swap, var. a, add.:%u, value:%d\n", &a, a);
7      printf("In swap, var. b, add.:%u, value:%d\n", &b, b);
8      printf("In swap, var. t, add.:%u, value:%d\n", &t, t);
9      t = a;
10     a = b;
11     b = t;
12 }
13 int main()
14 {   int x = 5, y = 7;
15     printf("In main, var. x, add.:%u, value:%d\n", &x, x);
16     printf("In main, var. y, add.:%u, value:%d\n", &y, y);
17     printf("Addr. of main(): %u\n", &main);
18     printf("Addr. of swap(...): %u\n", &swap);
19     swap (x, y);
20     printf("After swapping x and y\n");
21     printf("x=%d, y=%d\n", x, y);
22
23     system("pause");
24     return 0;
25 }
```

8 - Analyse a program to functions

Analyse a program to functions



9 - Implement a program using functions

Implement a program using functions

- ◆ Example: Develop a program that will print out the n first primes.

Analysis

- Nouns: the integer n → int n
- Verbs:
 - Begin
 - Accept n → simple
 - Print n first primes → function
 - End.

Result:

Input: n=5
Output: 2, 3, 5, 7, 11

Analysis

```
Function printNPrimes (int n)
    int count = 0;
    int value = 2;
    while (count < n)
    {
        if ( value is a prime → function
        {
            count = count +1;
            print out value; → simple
        }
        value = value +1;
    }
```

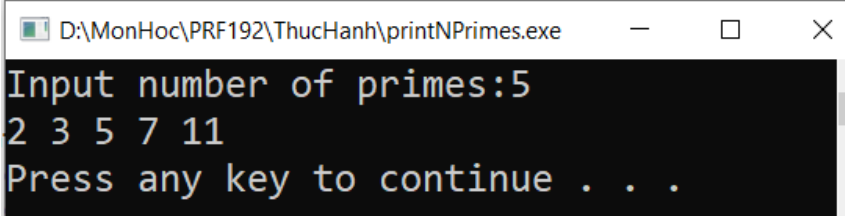
Implement a program using functions (cont.)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int isPrime(int n){
5      int i, flag = 1;
6
7      for(i=2; i*i<=n && flag==1; i++){
8          if(n%i==0){
9              flag=0;
10         }
11     }
12     return flag;
13 }
14
15 void printNPrimes(int n){
16     int count=0; // count primes printed
17     int value=2; // current value is considered
18     while(count<n){
19         if(isPrime(value)==1){
20             printf("%d ", value);
21             count++;
22         }
23         value++;
24     }
25 }
```

continue

```
27 int main(){
28     int n;
29     printf("Input number of primes:");
30     scanf("%d", &n);
31     printNPrimes(n);
32
33     printf("\n");
34     system("pause");
35     return 0;
36 }
```

Compile & Run



```
D:\MonHoc\PRF192\ThucHanh\printNPrimes.exe
Input number of primes:5
2 3 5 7 11
Press any key to continue . . .
```

Exercise 4

- ◆ Develop a program that will accept two positive integers then print out the greatest common divisor and the least common multiple of them.

- ◆ *Hint:*

Analysis

- Nouns: 2 integers $\rightarrow$ int m, n

The greatest common divisor $\rightarrow$ int G

The least common multiple $\rightarrow$ int L

- Verbs:

- Begin

- Accept m, n $\rightarrow$ simple

- G = Calculate the greatest common divisor of m,n $\rightarrow$ function **gcd**

- L = Calculate the least common multiple of m,n $\rightarrow$ function **lcm**

- Print out G, L $\rightarrow$ simple

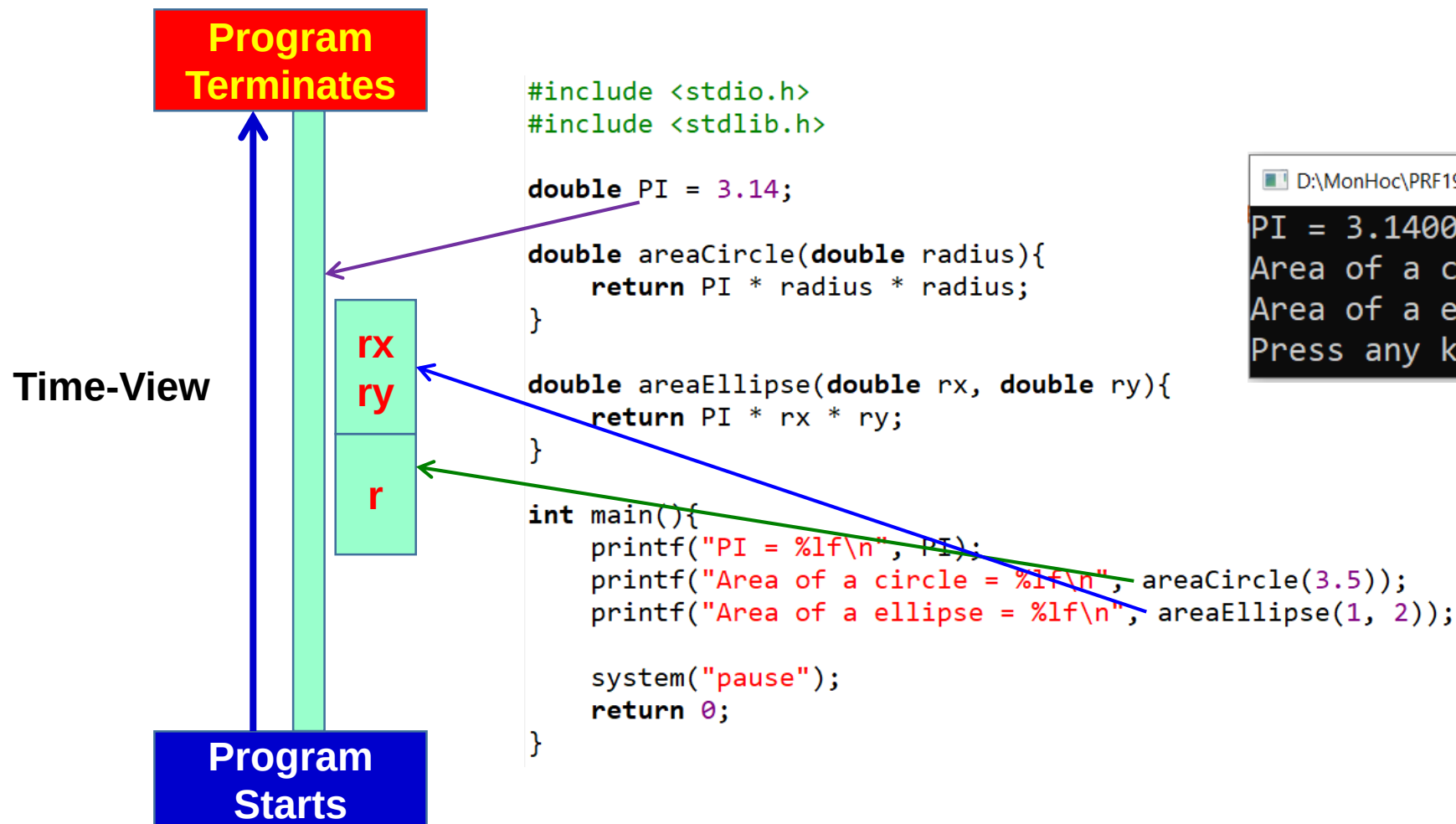
- End.

10 - Extent and Scope of a variable

Extent and Scope of a variable

- ◆ **Extent of a variable:** Duration begins at the time the memory of this variable is allocated to the time this block is de-allocated.
- ◆ **Scope of a variable:** The code block between the line which this variable is declared and the close brace of this block. In it's scope, the variable is visible (means that accessing to this variable is valid).
- ◆ **Global Variables:** Variables declared outside of all functions → They are stored in the data segment. If possible, do not use global variables because they can cause high coupling in functions.
- ◆ **Local Variables:** Variables declared inside a function → They are stored in the stack segment.

Extent of Variables: Time-View



```
D:\MonHoc\PRF192\ThucHanh\extent_var...
PI = 3.140000
Area of a circle = 38.465000
Area of a ellipse = 6.280000
Press any key to continue . . .
```

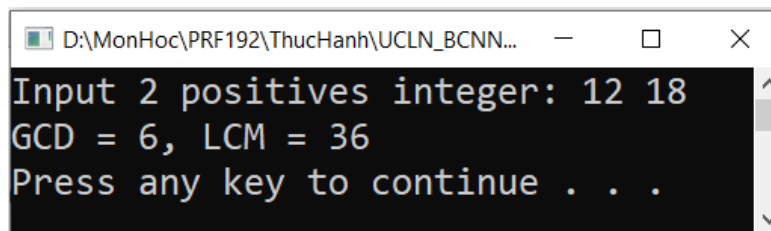
Scope of Variables: Code-View

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Find the greatest common divisor of two integers
5  int gcd(int a, int b) {
6      while (b != 0) {
7          int temp = b;
8          b = a % b;
9          a = temp;
10     }
11     return a;
12 }
13
14 // Find the Least common multiple of two integers
15 int lcm(int a, int b) {
16     return (a * b) / gcd(a, b);
17 }
18
19 int main(){
20     int m, n, G, L;
21
22     do{
23         printf("Input 2 positives integer: ");
24         scanf("%d%d", &m, &n);
25     }while(m<=0 || n<=0);
26
27     G = gcd(m, n);
28     L = lcm(m,n);
29
30     printf("GCD = %d, LCM = %d\n", G, L);
31
32     system("pause");
33     return 0;
34 }
```

Local variables of the function **gcd** include: memory containing return value (int), a, b

Local variables of the function **lcm** include: memory containing return value (int), a, b

Local variables of the function **main** include: memory containing return value (int), m, n, G, L



```
D:\MonHoc\PRF192\ThucHanh\UCLN_BCNN...
Input 2 positives integer: 12 18
GCD = 6, LCM = 36
Press any key to continue . . .
```

Extent and Scope of a variable: Visibility

```
#include <stdio.h>

int main ( ) {
    int input;

    printf("Enter a value : ");
    scanf("%d", &input);
    if ( input > 10) {
        int input = 5;  /* POOR STYLE */
        printf("The value is %d\n", input);
    }
    printf("The value is %d\n", input);

    return 0;
}
```

```
Enter a value : 12
The value is 5
The value is 12
```

Two variables have the same name (input) but they are different because the inner variable has the narrower scope than the outer variable → **RULE: Local first, Global later**

11 - Walkthroughs with Functions

Walkthroughs with Functions

- Given the following function and a case of using it. What is the value of the variable **t** when the function terminates?

```
int f( int a, int b, int c)
{
    int t= 2*(a+b-c)/5;
    return t;
}
```

```
int x = 5, y= 6, z= 7;
int t = 3*f(y,x,z);
```

y=6	x=5	z=7	f(a,b,c)
a	b	c	t
6	5	7	$2*(6+5-7)/5 = 1$

$t = 3*f(\dots) = 3*1 = 3$

Exercise 5

- Write a C program that will accept a non-negative integer then print out whether this number is power of 2 or not.

◆ *Hint:*

Analysis

- Variable: long n;
- Operation: Check a long integer n whether it is power of 2 or not (named isPower2)

```
return ((n & (n-1))==0);
```
- main function:
 Do
 accept n;
 While (n<=0)
 if (isPower2(n)==1) Print out “ It is power of 2”
 else print out “ It is not power of 2”

n	n binary	n&(n-1)
1	0000 0001	0000 0001 <u>0000 0000</u> 0000 0000
2	0000 0010	0000 0010 <u>0000 0001</u> 0000 0000
4	0000 0100	0000 0100 <u>0000 0011</u> 0000 0000
8	0000 1000	0000 1000 <u>0000 0111</u> 0000 0000
16	0001 0000	0001 0000 <u>0000 1111</u> 0000 0000

Exercise 6

- ◆ Write a C program that will
 - Accept 3 integers m, d, y that represent a date.
 - Print out they are valid or not.
 - Attention:
 - The February in a leap year will have 29 days.
 - If Y is a leap year then $(Y \% 4 == 0 \ \&\& \ Y \% 400 != 0) \ || \ (Y \% 100 == 0)$

Summary

- ◆ Module: A portion of a program that carries out a specific function and may be used alone or combined with other **modules** to create a program.
- ◆ Advantages of modules: It is easy to upgrade and it can be re-used
- ◆ C-function is a module
- ◆ A function is highly cohesive if all its statements focus to the same purpose
- ◆ Parameters make a function low coupling
- ◆ 4 parts of a function: Return type, function name, parameters, body
- ◆ Syntax for a function:

```
returnType functionName( Type param1, Type param2, ...)  
{  
    <<statements>  
}
```

Summary (cont.)

- ◆ Steps for implementing a function:
 - State the task clearly, verb is function name, nouns are parameters
 - Verb as find, search, calculate, count, check → return value function will return value.
Other verbs: void function
 - Give parameters specific values, do the work manually, write down steps done, translate steps to C statement
- ◆ Simple tasks: input/output some single value → Basic task → Library functions
- ◆ C-language uses the pass-by-value in passing parameters → The called function can not modify this arguments.
- ◆ Simple tasks: input/output some single values → Basic tasks → Library functions
- ◆ C-language uses the pass-by-value in passing parameters → The called function can not modify it's arguments.

Summary (cont.)

- ◆ Function prototype is a function declaration but its implementation is put at another place.
- ◆ Syntax for a function prototype:

returnType **functionName** (parameterType,...)

- ◆ Compiler will compile a program containing function prototype in three:
 - **Step 1:** Acknowledges the function template and marks places where this function is called.
 - **Step 2:** Update marks with function implementation if it is detected.
- ◆ Use a system library function: `#include<file.h>`
- ◆ Use **user-defined function in outside file**: `#include "filename"`
- ◆ Extent of a variable begins at the time this variable is allocated memory to the time this memory is de-allocated.
- ◆ Scope of a variable begins at the line in which this variable is declared to the closing brace containing it.